**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

# LESSON 084 · Bubble Sort

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

## Do Now · Recall previous learning

Q1. What must be true before binary search is used?

Q2. How can a subprogram make code easier to maintain?
A. Copy logic into every branch
B. Hide all useful names
C. Remove all testing
D. Change shared logic once

Q3. Why is indentation useful?

Q4. An unusual validation rule needs updating next year. “A comment changes the next variable's value.” Correct this. How could a comment help?

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

# I Do · Read new knowledge

## Key vocabulary

Bubble sort — Repeatedly compares and swaps adjacent values to order data.

Adjacent — Next to each other in the array.

Swap — Exchange positions of two values.

Pass — One traversal through consecutive pairs.

Ascending — From smallest to largest.

Temporary variable — Stores an element during an exchange to avoid losing it.

## New learning

Look at adjacent pairs: Bubble sort checks neighbouring elements from left to right, comparing a[i] with a[i+1].

Swap only if out of order: For ascending order, swap a pair when the left value is greater than the right value.

Complete one pass: Continue through pairs. Larger values move towards the rightmost end after a full pass.

Repeat to finish sorting: Repeat passes as necessary until all values are in order; an optimised version can stop when a pass makes no swaps.

## Worked example

Trace the first full ascending bubble-sort pass on [5,2,4]. State each swap and the resulting array. [4 marks]

1. Compare 5 and 2: 5 > 2 [1].

2. Swap to [2,5,4] [1].

3. Compare 5 and 4: 5 > 4 [1].

4. Swap to [2,4,5]; the pass is complete [1].

# We Do · Practise with guidance

### Guided 1

Trace the first ascending bubble-sort pass on [6,1,3]. Give the array after each swap. [4 marks]

Worked method: 6 > 1 so swap [1,6,3] [2]. Then 6 > 3 so swap [1,3,6] [2].

### Guided 2

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

For [2,4,3], explain whether the first pair swaps and what the pass produces. [3 marks]

Worked method: 2 < 4, so no swap [1]. Compare 4 and 3 and swap [1], producing [2,3,4] [1].

**Guided 3**

Explain why an array may need more than one bubble-sort pass. [2 marks]

Worked method: One pass moves larger elements rightwards [1] but smaller values can still be out of order and need further passes [1].

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

# You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

## Code First · Predict → Write → Test → Improve

Complete the missing comparison for one ascending bubble-sort pass.

```
array sizes = [6,1,3]
for i = 0 to 1
    if ... then
        temp = sizes[i]
        sizes[i] = sizes[i+1]
        sizes[i+1] = temp
    endif
next i
print(sizes[0])
print(sizes[1])
print(sizes[2])
```

write

## Question 1

State when two adjacent numbers swap during an ascending bubble sort. [2 marks] Source: Original OCR J277-style question; not an official OCR past-paper original.

## Question 2

Complete one bubble-sort pass on [7,2,5]. State the two intermediate arrays. [4 marks] Source: Original OCR J277-style question; not an official OCR past-paper original.

## Question 3

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Explain how bubble sort orders [4,3,1], showing the results after each necessary swap and pass. [6 marks] Source: Original OCR J277-style question; not an official OCR past-paper original.

## Exit Ticket · Answer before checking

Bubble sort compares neighbouring elements and may need several passes.

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

# Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

## Page 1 onwards · Do Now

Q1. The list must be sorted. This lets the search safely rule out half the remaining items.

Q2. D — Change shared logic once

Q3. Indentation shows blocks and nesting clearly. Some languages require it.

Q4. A comment explains why code works or what it is for. Clear comments help others understand and change it.

## I Do / We Do pages · Guided practice

Q1. 6 > 1 so swap [1,6,3] [2]. Then 6 > 3 so swap [1,3,6] [2].

Q2. 2 < 4, so no swap [1]. Compare 4 and 3 and swap [1], producing [2,3,4] [1].

Q3. One pass moves larger elements rightwards [1] but smaller values can still be out of order and need further passes [1].

## You Do pages · Code First

```
array sizes = [6,1,3]
for i = 0 to 1
    if sizes[i] > sizes[i+1] then
        temp = sizes[i]
        sizes[i] = sizes[i+1]
        sizes[i+1] = temp
    endif
next i
print(sizes[0])
print(sizes[1])
print(sizes[2])
```

Expected result: 1 3 6

## You Do pages · Independent questions

Q1. The left value is greater than the right value [1], so their positions are exchanged [1].

Q2. 7 > 2 [1], swap [2,7,5] [1]; 7 > 5 [1], swap [2,5,7] [1].

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Q3. Compare 4 and 3 [1], swap [3,4,1] [1]. Compare 4 and 1 [1], swap [3,1,4] [1]. Next pass compare 3 and 1 [1], swap [1,3,4], now sorted [1].

### Exit Ticket · Model answer

True

## Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

**1. Copy the whole original question:**

**2. Write a complete new answer in GREEN PEN. Show working if needed:**